

A Routed, Vectorizable Table Inverter for Machine-Precision Black–Scholes Implied Volatility

Wolfgang Schadner*

July 2026

Abstract

Recovering Black–Scholes implied volatility from an option price is among the most frequently executed numerical tasks in quantitative finance, and modern calibration, local-volatility, and surface-construction pipelines increasingly demand it at machine precision and at the throughput of vectorized hardware. We present a routed inverter that resolves the entire feasible domain by direct evaluation rather than by global iteration. The domain is decomposed into a central bivariate Chebyshev table in the transformed variable $W = h^2/2w$ together with three analytically matched charts—a small-moneyness matched expansion, an exact convergent large-volatility ladder, and a resurgent deep-wing inversion—whose special functions all reduce to a single two-parameter family. Each quote is routed by its IEEE-754 bit fields into branchless, fixed-operation-count kernels that the scalar and SIMD paths share identically, so batched evaluation reproduces the scalar result bit-for-bit. On every validation feed the AVX-512, AVX2, and non-SIMD builds, compiled with either GCC or Clang, return identical doubles. Against an independent 40-digit oracle the worst relative error in volatility is 8.3×10^{-16} across the whole domain, matching the reference iterative solver where both attain machine precision and holding that precision in the near-intrinsic and deep-wing regimes where it degrades. All four charts vectorize. A speculative streaming driver, built around the empirical fact that a real option book is about 90% small-moneyness quotes, inverts a market-realistic S&P 500 feed at 2.1–2.9 times the throughput of the reference and fixed-maturity surfaces at 3–9 times, at bit-identical accuracy. Every representable input receives either a machine-precision result or a flagged non-result. On the tradeable book both solvers are at machine precision, so the case for the method there rests on throughput and determinism, with the accuracy advantage serving as insurance on the extreme inputs a robust system must also survive.

1 Introduction

The implied volatility of a European option is the value of the volatility parameter that equates the Black–Scholes price to an observed market price. Because it is the coordinate in which option prices are quoted, compared, interpolated, and risk-managed, its computation is, in the words of Jäckel (2006), at once one of the most frequently executed and one of the least documented numerical tasks in derivatives modelling. A single recalibration of a volatility surface, a local-volatility bootstrap, or a Monte Carlo pricing grid can require millions of inversions, and each must be accurate: an inverter that is correct only to a few digits injects noise into every downstream Greek and every calibrated parameter. The workload shape matters as much as the count. Production systems rarely invert one quote at a time; they invert a strip of strikes at one maturity, a whole surface, or a dense grid, and on modern hardware the difference between a method that processes such batches eight lanes at a time and one that cannot is a larger factor than any remaining difference in single-quote cost. That batch axis, not scalar latency, is where the present work aims.

*University of Liechtenstein, wolfgang.schadner@uni.li

Two largely separate literatures address the problem. The first seeks explicit, closed-form approximations to the inverse map. Li (2008) fits rational functions of moneyness and maturity; Stefanica and Radoičić (2017) derive an explicit formula with a uniform relative-error bound, and Xia and Cui (2018) an explicit series inversion. These are fast and naturally vectorizable, but they hold only on a bounded region of the input plane and only to modest precision. The second literature refines an initial guess to full machine precision. The reference method here is Jäckel (2015)’s *Let’s Be Rational*, which combines a four-branch rational seed with two third-order Householder iterations on a carefully reformulated normalized Black function, and attains the last representable bit throughout the domain. Its accuracy is not in question; the cost is that a branch-rich, data-dependent iteration is difficult to vectorize, and its natural unit of work is a single scalar quote. The most recent entrant, Le Floc’h and Healy (2026), keeps the seed-plus-refinement structure but engineers it for low latency, trading a small, controllable amount of reference-price alignment for speed. A third, smaller strand precomputes the inverse map itself: Matić et al. (2018) fit a polynomial table by a PDE method for batch-oriented use, but target an accuracy of 10^{-7} , eight orders of magnitude short of the last bit.

The gap this leaves is a method that reaches machine precision *and* vectorizes over the whole feasible domain, so that the batched workloads which dominate production—an entire surface at one maturity, or a mixed grid of quotes—run at hardware throughput rather than at scalar-iteration latency. We close that gap by treating inversion not as “seed and refine” but as *routing and direct evaluation*. The feasible region is partitioned into four regimes, each carrying an approximation in the coordinate natural to it: a central bivariate Chebyshev table in the transformed variable $W = h^2/2w$, whose cell geometry and polynomial degree are fixed by the analytically known, maturity-independent location of the inverse map’s branch points; a matched small-moneyness expansion built on the exact at-the-money kernel; an exact, absolutely convergent large-volatility ladder; and a resurgent deep-wing inversion. A central structural result is that the special functions of all four charts are one two-parameter family, so the entire construction rests on a single evaluator plus the inverse error function. Every quote is routed into its regime by integer operations on its IEEE-754 bit fields, and evaluated by branchless kernels written so that the scalar entry point and the SIMD batch drivers execute the identical sequence of fused multiply-adds per quote. The batch result is therefore bit-for-bit identical to the scalar result, which we treat as a non-negotiable correctness invariant and verify exhaustively; enforcing it turns out to require a compiler discipline (Section 5) whose necessity we can demonstrate, and which we believe is not widely appreciated: under GCC’s default floating-point contraction the same header can compile to two functions that differ in the last bit in two translation units of one program.

Against an independent high-precision oracle the method attains a worst-case relative error in volatility of 8.3×10^{-16} across the full feasible domain, matching Let’s Be Rational wherever both are at machine precision and holding that precision in the two extreme regimes where the reference measurably degrades, and it does so with a fixed evaluation cost per quote: every chart, including the two that finish with a short refinement, executes a fixed operation count with no data-dependent iteration. All four charts vectorize, the scalar and SIMD builds of both instruction sets produce bit-identical results under either of two compilers, and every representable input pair receives either a machine-precision result or an explicitly flagged non-result, a totality property we regard as part of what makes an inverter reference-grade. We also ask what a *realistic* workload exercises: routing an entire year of tradeable S&P 500 index option quotes through the classifier shows the small-moneyness chart carrying about 90% of a live book and the deep wing a further 5% (Section 6). That concentration is not a nuisance but a lever: a speculative streaming driver evaluates the dominant chart on every lane and stores only the ones that belong, sorting only the minority, and inverts a market-realistic S&P 500 feed at 2.1–2.9 times the reference’s throughput (AVX2 and AVX-512) and fixed-maturity surfaces at 3–9 times, at bit-identical accuracy, and in the streaming regime a live system runs its re-inversion at five

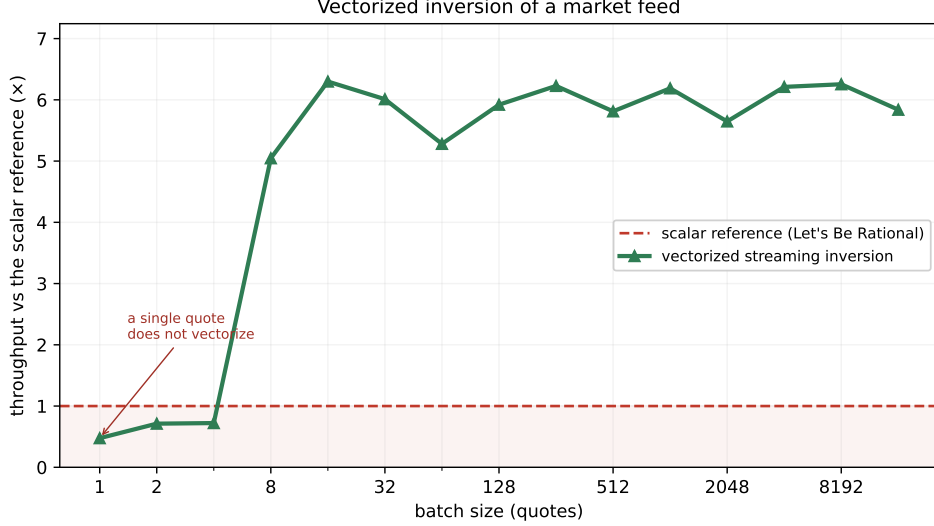


Figure 1: The central performance claim, on real data. Inverting a 2024 S&P 500 book, the vectorized streaming re-inversion of Section 5 sustains about five to six times the per-quote throughput of the scalar reference solver, flat in batch size once a batch fills one SIMD register. A single isolated quote cannot be vectorized and is the one regime the scalar reference still wins. Throughput is measured against the reference in the same session, so the host clock cancels.

to six times the reference (Figure 1). On that same tradeable book both solvers reach machine precision, so the case for the method on real data is throughput and determinism, with the accuracy advantage held in reserve for the extreme inputs a robust deployment must also handle. The remainder of the paper is organized as follows. Section 2 fixes the normalized inversion problem and notation. Section 3 constructs the central table and the branch-point argument that sizes it. Section 4 develops the three endpoint charts and their unification, and states the accurate-forward evaluation that sets the precision floor. Section 5 describes the routing and the scalar/SIMD bit-identical implementation. Section 6 reports accuracy and performance against an independent oracle and against Let’s Be Rational. Section 7 concludes.

2 The normalized inversion problem

We work throughout with the normalized out-of-the-money (OTM) Black price. Writing $h = |\log(K/F)| \geq 0$ for the absolute log-moneyness, $w = \sigma^2 T$ for the total variance and $v = \sqrt{w} = \sigma\sqrt{T}$ for the total standard deviation, the normalized OTM price is

$$c = \Phi\left(\frac{v}{2} - \frac{h}{v}\right) - e^h \Phi\left(-\frac{v}{2} - \frac{h}{v}\right) \in (0, 1), \quad (1)$$

with Φ the standard normal cumulative distribution function. Calls and puts map to the same problem by the put–call symmetry $h \mapsto -h$, so it suffices to treat $h \geq 0$; the inversion problem is to recover w (equivalently σ) from a given pair (h, c) . This is the same reduction used by Jäckel (2015) and Le Floc’h and Healy (2026), save that we normalize the price by the forward F rather than by $\sqrt{FK}$, a choice that matters in the deep wing (Section 6).

All of the analysis below flows from a single integral representation of (1). Differentiating the Black price with respect to the total standard deviation and integrating back gives the exact form

$$c = C(h, v) = k e^{h/2} \int_0^v e^{-t^2/8} e^{-h^2/(2t^2)} dt, \quad k = \frac{1}{\sqrt{2\pi}}, \quad C(h, \infty) = 1, \quad (2)$$

in which the integrand is the product of a Gaussian factor $e^{-t^2/8}$, regular at $t = \infty$, and a boundary-layer factor $e^{-h^2/(2t^2)}$ that vanishes to all orders at $t = 0$. The two factors are the two endpoints of the inversion problem. The layer at $t = 0$ governs the small-price wing, and the regular tail governs the large-volatility limit $c \rightarrow 1$; the interior, where neither factor simplifies, is where a table is the natural device. Section 3 treats the interior and Section 4 the two endpoints.

The map (1) is analytic and strictly monotone in v , but it is numerically treacherous at both ends of the price range. As $v \rightarrow 0$ the OTM price collapses like $c \sim \varphi(h/v) v/h$, so a plain Newton iteration on $c - C(h, v)$ has a vanishing derivative and loses all its significant digits; as $v \rightarrow \infty$, $c \rightarrow 1$ and the two terms of (1) cancel. Robust inversion therefore requires working in a variable in which the map is well conditioned. The organizing object is the endpoint factorization of (1): with the exponential scale variable

$$W = \frac{h^2}{2w} = \frac{h^2}{2v^2}, \quad (3)$$

one has the exact representation $c = C(h) e^{-W} W^{-3/2} H(W)$, in which $C(h) = \frac{h}{2} e^{h^2/2} / (2\sqrt{\pi})$ and H is an elementary, error-function-free convergent integral with $H(W) \sim 1 - (a + \beta)/W + \dots$ for a fixed pair $(a, \beta) = (3/2, h^2/16)$. This factorization is the deep-wing form of a generalized-inverse-Gaussian quantile representation of the Black map; viewed in the complex price plane, the map extends analytically to an annulus whose boundary circles pass through the two branch points $c = 0$ and $c = 1$, a geometric fact that returns in Section 3 as the driver of the table's polynomial degrees. The variable W is the one in which the small-price behaviour is smooth, and it is the variable the central table stores.

It is worth recording at the outset what accuracy is even attainable. Writing $\nu(h, v) = \partial C / \partial v$ for the normalized vega, a relative perturbation ε of the input price moves the exact root by the relative amount $\kappa \varepsilon$ with $\kappa = c/(v \nu)$, the relative condition number of the inversion. From (2), deep in the wing $\log c \approx -W$ so $\kappa \approx 1/(2W)$, which is *small*: the wing is benignly conditioned in relative terms, and there is no obstruction to recovering σ to machine precision from prices as small as the smallest positive doubles. Near the upper endpoint the price itself is the wrong variable, since $\kappa \rightarrow \infty$ as $c \rightarrow 1$; but the *complement* $1 - c$ is again benignly conditioned, with a relative perturbation of $1 - c$ moving the root by only $\mathcal{O}(v^{-2})$ times as much. The design consequence, which recurs throughout the paper, is that full accuracy is attainable everywhere provided every formula works with whichever of c and $1 - c$ is small, and never manufactures the small one by subtraction. The reference solver of Jäckel (2015) makes the same observation for its own normalization; the residual accuracy differences reported in Section 6 trace precisely to where each method honours it. One bound, however, no algorithm can cross: a caller who passes the price as a plain double has already rounded $1 - c$, and once $v \gtrsim 8$ that rounding alone moves the root by more than 10^{-13} , growing to 10^{-3} near $v \approx 16.6$ where $1 - c$ reaches machine epsilon. This is a property of the input representation, shared by every double-precision inverter including the reference; the implementation surfaces it honestly through a status flag rather than refusing the input (Section 5).

We score accuracy as the relative error in σ against an independent arbitrary-precision oracle (Section 6), never against a round trip through a forward pricer. This distinction is essential: the forward map (1) loses precision to cancellation near $c \rightarrow 0$ and $c \rightarrow 1$, so re-pricing a computed σ and comparing conflates forward-pricer error with inverter error and can inflate the apparent error of *any* correct inverter by two orders of magnitude in the wings. The oracle instead inverts the *given* double-precision price c at 40 significant digits.

3 The central table

Over the interior of the domain the inversion is resolved by direct evaluation of a precomputed table, with no iteration at all. The rationale is a cost observation before it is an accuracy

one. A machine-precision iterative inverter spends essentially all of its time in its accurate forward evaluations: a seed plus two Householder corrections costs two evaluations of a carefully written Black function, each dominated by error-function calls. A table evaluation replaces all of that by one polynomial sweep whose operation count is fixed, known at compile time, and free of data-dependent branches, which is also exactly the shape of computation that vectorizes (Section 5). Precomputing tables is, moreover, standard deployment practice in latency-sensitive trading systems; the questions that decide whether the approach reaches machine precision are which variable to tabulate, in which coordinate, and at what polynomial degree. All three answers come from the analytic structure of the inverse map.

The approximation target is the transformed variable W of (3) rather than σ or w themselves: W is the exponential scale variable of the wing asymptotics, it is nearly linear in $\log c$ across each cell, and fitting σ directly was measured to cost one to two additional Chebyshev degrees per cell for no benefit. The abscissa is $\lambda = \log c$. The reason λ is the right coordinate, and the reason no cleverer coordinate can do better, is a statement about where the inverse map ceases to be analytic. Viewed as a function of the price c at fixed h , the inverse $w(h, \cdot)$ has its only real-axis singularities at the two branch points $c = 0$ and $c = 1$, and, once the annulus geometry of Section 2 is accounted for, the distances from an interior price c to these branch points are exactly c and $1 - c$, *independently of h* : the maturity dependence cancels. The convergence of a Chebyshev approximation on a cell is governed by the Bernstein ellipse through the nearest singularity, so this one fact fixes the degree demanded by every candidate coordinate. In the raw mantissa (binade) coordinate the $c = 0$ branch point sits one octave to the left of every octave, in a self-similar way, which pins the Bernstein parameter at $\rho = 3 + 2\sqrt{2}$ and forces a floor of roughly twenty-one degrees per octave everywhere; a transcendental-free table in that coordinate is therefore a dead end, elegant as it looks. A conformal coordinate adapted to the annulus fares even worse in practice (fitted degrees near one hundred), because it compresses the real interval against the boundary. In $\lambda = \log c$ the distance to the branch point grows with the logarithmic cell width, and the fitted degrees fall to between six and thirteen. These are measured degrees from the table generator, not estimates, and they were the basis on which the coordinate was frozen.

On each cell the table stores a bivariate Chebyshev surrogate

$$W \approx \sum_{i=0}^{d_h} \sum_{j=0}^{d_\lambda} c_{ij} T_i(x_h) T_j(x_\lambda), \quad (4)$$

with $x_h, x_\lambda \in [-1, 1]$ affine images of the local h -band and $\log c$ interval. The degree never exceeds sixteen in the price direction and thirteen in the maturity direction, with median near ten. The full table comprises nineteen h -bands over $h \in [0.30, 6.65]$ and 181 cells holding about seventeen thousand double-precision coefficients, just under 200 KB in total: small enough to be resident in a single core's L2 cache, so that in steady state the table costs no memory traffic. Cache-cold and cache-warm evaluation were timed separately during development and are indistinguishable after the first few hundred quotes of a batch, so table residency is a realized property rather than a hope. Recovery from the surrogate is the single map $w = h^2/(2W)$.

Routing to a cell is performed by integer operations on the IEEE-754 bit fields of the inputs, before any transcendental evaluation. The high bits of h (exponent plus two leading mantissa bits) select the h -band; the exponent bits of c select a price octave within the band; and the mantissa of c , forced into $[1, 2)$, becomes the evaluation abscissa through the shared logarithm kernel of Section 5, so that the per-quote classification involves no division, no logarithm and no comparison ladder. The octaves adjacent to $c = 1$, where the near-at-the-money geometry would otherwise demand a special analytic branch, are instead refined by one or two additional mantissa bits into sub-cells, so that they become ordinary table cells; this sub-splitting is what removes the last analytic bypass from the interior. The whole classification is branch-light and identical in the scalar and vectorized paths.

The bands deliberately stop at $H_{\text{box}} \approx 6.65$. Beyond it the feasible strip is owned entirely by the endpoint charts of Section 4: for such large h every feasible quote is either deep in the wing (small price) or above the $v = 2$ seam (large volatility), so extending the table would add cells that no quote needs. The same reasoning caps the tabulated maturity range from below at $h = 0.3$, where the matched small-moneyness chart is uniformly accurate. The table is therefore exactly as large as the region in which neither endpoint structure simplifies, and no larger.

The table is generated offline and certified before it is frozen. Chebyshev nodes for each cell are obtained by inverting exact double-precision prices with an arbitrary-precision solver at no fewer than twenty-five significant digits; the tensor coefficients are fitted; and the cell is accepted only if an *independent* forty-digit oracle, distinct from the generator, confirms a relative σ error below 10^{-15} on a dense sample of the cell including its boundaries. Cells failing the gate are re-fitted at higher degree or split. The same oracle-gated discipline applies to every auxiliary kernel of Section 5; one lesson from it is recorded there, because it is easy to get wrong: for a kernel feeding a downstream fit, the *smoothness* of the approximation error matters as much as its size.

4 The endpoint charts and the accurate forward

The table covers the interior; three analytically matched charts cover the endpoints, and together they cover the entire feasible domain with no additional two-dimensional tables. The charts are constructed so that each is most accurate exactly where the others fail, and the complementarity is exact rather than incidental: it mirrors the structure of the base integral (2), whose boundary layer at $t = 0$ is resolved by a matched coordinate, whose regular Gaussian tail is resolved by an elementary price transform, and whose middle, where neither simplification applies, is the table's natural home.

4.1 The small-moneyness chart

For h below the threshold $h < 0.3$ the geometry collapses toward the explicit at-the-money line $c = \text{erf}(\sigma/2\sqrt{2})$: the level sets of c crowd against an exactly known curve, and direct two-dimensional approximation of $w(h, c)$ degenerates precisely where the answer is simplest. The cure is a matched coordinate that absorbs the collapse. Scale the price by the leading h -dependence,

$$\rho = \frac{c}{\text{expm1}(h)}, \quad (5)$$

and let $A = B^{-1}(\rho)$ where

$$B(A) = \frac{\varphi(A)}{A} - \Phi(-A) = \int_A^\infty \varphi(u) u^{-2} du \quad (6)$$

is an h -free, strictly decreasing universal function arising from the $t \rightarrow 0$ layer of (2): the substitution $t = h/u$ maps the layer factor $e^{-h^2/(2t^2)}$ to the Gaussian $\varphi(u)$ and produces exactly the integral B , so the coordinate is dictated by the integral representation rather than chosen. The matched variable is $s = h/A$, which reduces to v as $h \rightarrow 0$, and in it the volatility has an even, regular expansion

$$\sigma(h, c) = \sigma_0(\kappa s) + h^2 V_2(s) + h^4 V_4(s) + h^6 R(s, h), \quad (7)$$

where $\sigma_0(\cdot) = 2\sqrt{2} \text{erf}^{-1}(\cdot)$ is the exact at-the-money kernel, $\kappa = 1/\sqrt{2\pi}$, and V_2, V_4 are elementary functions of s alone. Evenness in h is not an accident of fitting but the put-call symmetry $h \mapsto -h$ of the underlying density, and it is what makes the expansion efficient: each retained order gains a uniform factor h^2 . This was verified directly, by halving h at fixed v and observing the residual error of the one-, two- and three-term truncations divide by 4, 16 and 64.

The leading term alone is uniformly second-order accurate through the deep-OTM corner (small c jointly with small h) that defeats naive tabulation, which is the region where the predecessor iterative method exhibited its only accuracy defect.

The chart is completed by two implementation decisions. First, the residual R is small enough to be frozen once as a single low-order bivariate Chebyshev surface (degree two in the h^2 direction, sixteen in s); it absorbs everything beyond h^4 and brings the chart to machine precision across its whole strip. Second, the two functions V_2 and V_4 , whose closed forms contain exponentials and a series crossover, are replaced by one-dimensional Chebyshev expansions in s of degrees 28 and 26; this removes the last library call and the last data-dependent branch from the chart's inner loop, which matters for the vectorized path of Section 5. The universal inverse B^{-1} itself is evaluated by a three-regime branchless Chebyshev approximation, one regime in the variable $u = \kappa/(\rho + \frac{1}{2})$ for moderate arguments and two in $L = \log(1/\rho)$ for the tail, totalling 424 bytes of coefficients with worst error 5.4×10^{-16} ; the chain $c \mapsto \rho \mapsto A \mapsto s$ has relative condition number at most one, so no accuracy is lost in the transforms.

4.2 The large-volatility chart

At the opposite endpoint, $v > 2$ and $c \rightarrow 1$, the complement of (2) is

$$1 - c = k e^{h/2} \int_v^\infty e^{-t^2/8} e^{-h^2/(2t^2)} dt, \quad (8)$$

and the structural point that makes this endpoint fundamentally easier than the left one is that the layer factor is *regular* there: $e^{-h^2/(2t^2)} \rightarrow 1$ as $t \rightarrow \infty$, with an expansion in h^2/t^2 that converges absolutely for every $t > 0$. There is no layer to match, and termwise integration (justified by dominated convergence) yields an exact, absolutely convergent series rather than an asymptotic one. Introducing the ladder $B_n(A) = \int_A^\infty \varphi(u) u^{-2n} du$, which satisfies the elementary recursion $B_{n+1}(A) = [\varphi(A)A^{-(2n+1)} - B_n(A)]/(2n+1)$ with $B_0(A) = \Phi(-A)$ and $B_1 = B$, one obtains with $x = v/2$

$$\frac{(1-c)e^{-h/2}}{2} = \Phi(-x) + \sum_{n \geq 1} \frac{(-h^2/8)^n}{n!} B_n(x). \quad (9)$$

Every identity in this subsection was checked against 30-digit arithmetic at scattered (h, v) up to $h = 6.6$, $v = 7$; agreement is exact to working precision.

Two consequences follow immediately. Setting $h = 0$ in (9) identifies the leading inverse as the *same* at-the-money kernel as on the left, evaluated at the transformed price

$$\tilde{c} = 1 - (1 - c) e^{-h/2}, \quad v \approx 2\sqrt{2} \operatorname{erf}^{-1}(\tilde{c}) + h^2 V_2^R + h^4 V_4^R + \dots, \quad (10)$$

where $V_2^R = -B_1(x_0)/(4\varphi(x_0))$ and the higher coefficients are, through the ladder recursion, polynomials in $1/x_0$ and the Mills ratio $m(x) = \Phi(-x)/\varphi(x)$; the Mills ratio is the only genuinely new one-dimensional function the endpoint requires. And since $B'_n(x) = -\varphi(x)x^{-2n}$, the series for the derivative of the right-hand side of (9) resums in closed form,

$$g'(x) = -\varphi(x) e^{-h^2/(8x^2)}. \quad (11)$$

The truncated ladder alone is excellent deep in the endpoint, where the expansion parameter $W = h^2/(2v^2)$ is small, but degrades toward moderate v ; Table 1 reports the verified relative error of the three-term truncation across the strip. The chart is therefore finished by iterating on (9) itself: the equation is exact and the derivative (11) is cancellation-free and free of error functions. Moreover, because $\log|g'|$ is the elementary expression $\operatorname{const} - x^2/2 - h^2/(8x^2)$, the higher derivative ratios needed by a third-order Householder step are closed-form rationals, $g''/g' = -x + h^2/(4x^3)$ and $g'''/g' = (g''/g')^2 - 1 - 3h^2/(4x^4)$, so an order-four step costs the same

h	v	$W = h^2/(2v^2)$	rel. error of $V_0^R + h^2V_2^R + h^4V_4^R$
0.3	7.0	0.0009	1.8×10^{-12}
1.0	7.0	0.0102	2.4×10^{-9}
2.0	7.0	0.0408	1.5×10^{-7}
6.6	7.0	0.44	1.7×10^{-4}
2.0	2.5	0.32	2.0×10^{-3}

Table 1: Verified relative accuracy of the three-term ladder seed of the large-volatility chart, before the Householder finisher; the expansion parameter is $W = h^2/(2v^2)$. The finisher’s exact equation and closed-form derivatives bring every point to machine precision.

one price evaluation as a Newton step. This is the same economics that motivates Householder refinement in Jäckel (2015), and it halves the finisher: the implementation performs a *fixed* count of three Householder steps with no early exit. The count was sized by measuring pure iteration convergence against exact prices: the seed degrades to a relative error of 0.27 at the worst corner of the strip (near the wing seam, $W \approx 3$), from which the fourth-order steps contract through 10^{-2} , 10^{-7} and 10^{-27} , so the third step lands with eleven orders of margin while two steps demonstrably do not suffice. A fixed count costs a little at benign points but buys determinism and lane-uniformity in the vectorized path. Prices above one half are handled throughout in the complement form, so that $1 - c$ is never manufactured by subtraction; this, with the cancellation-free transform $c \mapsto \tilde{c}$, is why the chart retains full precision as $c \rightarrow 1$ where a $\sqrt{FK}$ -normalized formulation measurably does not (Section 6).

4.3 The deep wing

For prices below the table’s reach the inversion uses the resurgent form of the endpoint factorization of Section 2. Taking logarithms of $c = C(h)e^{-W}W^{-3/2}H(W)$ turns the inversion into the implicit equation

$$W = \tilde{L} - \frac{3}{2} \log W + \log H(W), \quad (12)$$

where $\tilde{L}$ collects the price and h data. Because the equation lives on the logarithm of the price, its conditioning is the benign $1/(2W)$ of Section 2, and the inverter is validated on prices down to 10^{-320} , i.e. across the entire subnormal range of double precision; the shared logarithm kernel of Section 5 is extended by an exact power-of-two rescaling so that subnormal prices lose no accuracy on entry.

What makes the chart vectorizable is the evaluation of $\log H$. A runtime quadrature, the natural first implementation, has neither a fixed operation count in the useful sense (forty exponentials per evaluation) nor competitive speed, so it is retired to the offline oracle role and replaced by a frozen two-piece bivariate Chebyshev fit. The fitted quantity is deliberately not $\log H$ itself but the peeled, weighted residual

$$S(u, \beta) = \frac{\log H}{u} + \left(\beta + \frac{3}{2}\right), \quad u = \frac{1}{W}, \quad (13)$$

whose two transformations each earn their place: peeling the leading asymptotic term $-(\beta + 3/2)/W$ removes the dominant variation, and dividing by u makes the fit’s equioscillating error automatically proportional to the σ -tolerance, which grows like W ; fitting the raw residual instead was measured to demand degrees beyond thirty-six where the weighted form needs twenty-six. Two pieces with a generous overlap cover $W \in [2.5, 260]$ (668 coefficients, 5.4 KB), the eight-term asymptotic series takes over beyond, and the piece-or-series regime is selected *once* per quote from $\tilde{L}$, so no per-step branching enters the kernel. The derivative $\partial_W \log H$ that drives the Newton steps is computed from the *same* coefficients by the Chebyshev derivative recurrence, so the value and slope seen by the iteration are consistent by construction. The

solve is a fixed count of six Newton steps. Five were originally de-risked on a sparse grid, and a dense sweep later exposed the choice as one step short at the high- β , low- W corner, where convergence is slowest; the incident is recorded in Appendix B because it illustrates the failure mode of sparse step-count de-risking. With six steps the chart passes the 10^{-15} oracle gate at every point of every feed, and it now shares the fixed-operation structure of the other charts, so the batch drivers evaluate it eight lanes at a time like the rest.

One boundary of the fitted table must be handled rather than clamped. The fit covers $\beta = h^2/16 \leq 16.4$, i.e. $h \leq 16.2$, corresponding to strikes more than 10^7 times the forward; benchmark grids nevertheless reach past it, and a clamped evaluation there would return a plausible-looking volatility wrong in the third digit, the worst kind of failure. Quotes beyond the fitted β therefore take a rare scalar branch: a seed obtained by dropping the second Black term and solving $\Phi(v/2 - h/v) = c$ in closed form (then restoring the dropped term by three fixed-point passes, worst seed error 0.23% over $h \in [16.2, 30]$), finished by the same three Householder steps as the warm-start entry. The branch is certified for $W \lesssim 40$; the still-deeper corner, unreachable by any market data, returns a quiet NaN with an explicit out-of-domain status rather than a number (Section 5). Both the scalar router and the batch drivers call the identical branch function, so the bit-identity invariant is preserved structurally.

4.4 One family, certified bounds, and derived seams

A single identity ties the charts together. With

$$H(W; a, \beta) = \int_0^\infty e^{-r} \left(1 + \frac{r}{W}\right)^{-a} e^{-\beta/(W+r)} dr, \quad (14)$$

the substitution $u^2 = A^2 + 2r$ gives, exactly,

$$B_n(A) = \varphi(A) A^{-(2n+1)} H\left(\frac{A^2}{2}; n + \frac{1}{2}, 0\right), \quad (15)$$

and the same family evaluated at $(a, \beta) = (3/2, h^2/16)$ and $(1/2, h^2/16)$ furnishes the two deep wings. Every special function appearing anywhere in the architecture is therefore one two-parameter family, and only two genuinely one-dimensional, h -independent inverses (erf^{-1} and B^{-1}) ever appear; in matched coordinates, all the h -dependence is a smooth prefactor times a regular even perturbation. For $\beta = 0$ the family is Stieltjes,

$$H(W; a, 0) = \frac{1}{\Gamma(a)} \int_0^\infty \frac{t^{a-1} e^{-t}}{1 + t/W} dt, \quad (16)$$

so its continued-fraction convergents bound it from alternating sides, and B , every B_n , and by monotone inversion B^{-1} admit certified two-sided enclosures at any order; this is the foundation on which the endpoint kernels were certified. For $\beta > 0$ the Stieltjes property fails (the Borel transform of the wing series changes sign, and the Herglotz–Pick criterion is violated numerically), but an exact integral identity expresses $H(\cdot; a, \beta)$ through the $\beta = 0$ family, so certified enclosures extend to the wing evaluators as well.

The seams between the charts are derived, not tuned. The truncation error envelopes of the two endpoint expansions obey the verified scaling laws $E_{\text{left}} \propto v^2 h^{2N+2}$ and $E_{\text{right}} \propto (h^2/2v^3)^{N+1}$, and equating envelopes across the wide overlap bands, where two charts are simultaneously accurate, yields thresholds that are constants in the matched coordinates. The implementation freezes all three as one-dimensional price curves, each a single Chebyshev polynomial in h : the wing seam $c_{\text{wing}}(h) = C(h, h/\sqrt{6})$ (the level set $W = 3$), the central–right seam $c_2(h) = C(h, 2)$ (the level set $v = 2$), and, within the small- h band, the left–right seam $c_{\text{tl}}(h) = C(h, 1.70)$ —placed at $v = 1.70$ rather than $v = 2$ so that the exact-equation Householder refinement of Section 4.2, rather than an extrapolated finisher, owns the corner where the two charts meet. Freezing

the seams as polynomials matters beyond tidiness: routing then costs three short polynomial sweeps rather than any runtime price evaluation, which is what allows the classification itself to vectorize (Section 5). A fitted seam differs from the exact boundary only within its fit error ($\sim 10^{-16}$ relative), reclassifying at most quotes within a ulp of the seam—inside the overlap bands, where both neighbouring charts are machine-precise, so nothing delicate happens at a crossing; the dense seam sampling reported in Section 6 confirms it.

The resulting coverage map of the feasible domain is worth stating in one place. The deep wing owns $W \gtrsim 3$, including the feasibility pinch at very large h , which is a wing region in disguise since the feasible price there is necessarily small. The small-moneyness chart owns $h < 0.3$ up to the $v = 1.70$ seam. The large-volatility chart owns everything above its seam, including the whole strip $h > H_{\text{box}}$ that is not wing. The central table owns the remainder. The four regimes tile the domain exactly: on a broad random feed no feasible quote fails to route, and the formerly excluded small- h deep-OTM corner of the predecessor design is now interior to the small-moneyness chart.

The accurate forward. As in Jäckel (2015), the precision floor of the whole method is the accuracy of the forward evaluation used inside the places that refine, the large-volatility Householder finisher and the warm-start entry of Section 5. We evaluate the normal functions there through a scaled complementary error function `erfcx` built on frozen Chebyshev pieces in the spirit of Cody (1969)—three degree-22 pieces covering $[0, 7.2]$, selected per lane by blends off the critical path, which halves the sequential Clenshaw chain relative to a single degree-36 fit and extends the domain to the arguments the warm-start basin requires—and an exponential reduced by a Cody–Waite splitting of $\log 2$ followed by a Chebyshev polynomial on $[-\frac{1}{2} \log 2, \frac{1}{2} \log 2]$ and an exact power-of-two scaling. Both Black terms of a price residual are produced by one shared kernel that evaluates the $\pm$ pair from a single `erfcx` core and one shared normal tail, with a compensated square killing the rounding of the squared argument, so each refinement step costs two `erfcx` and four exponential evaluations rather than the naive six. The refiners therefore perform no library `erf`, `erfc` or `exp` call, their rounding behaviour is fully determined by the frozen coefficients, and the rational quantile seed’s 10^{-9} error is immaterial because the equation being solved is exact.

5 Routing and the bit-identical vectorized implementation

The design goal that shapes the implementation is that a batched evaluation must return, for every quote, the same double-precision result as the scalar entry point, bit for bit. This is not a convenience: it means the vectorized surface and grid drivers can be validated by the single statement “batch equals scalar”, and it removes any possibility that the fast path and the reference path silently disagree. The invariant is achieved by writing every kernel as a branchless chain of *explicitly ordered* fused multiply-adds and sharing that exact code between the scalar and SIMD paths.

Three kernels are shared. A branchless logarithm proxy evaluates $\log_2 m$ for a mantissa $m \in [1, 2)$ through the reduction $t = (m - 1)/(m + 1)$ and a Chebyshev series in t^2 , with absolute error a few units in the last place, comparable to the library function it replaces; a full natural logarithm, needed by the universal inverse, the quantile seed and the wing, is then assembled by reuse as $\log x = \log 2 (e + \log_2 m)$ from the exponent e and mantissa of x , introducing no new approximation and agreeing with the library logarithm to three units in the last place over the domains that consume it; a subnormal-safe variant rescales arguments below the normal range by an exact power of two before the bit extraction, so the wing’s prices near 10^{-320} enter with full accuracy. A degree-seventeen branchless kernel evaluates the at-the-money base $\sigma_0 = 2\sqrt{2} \operatorname{erf}^{-1}$. And a bivariate Chebyshev evaluator computes (4) by an inner accumulation in x_λ followed by an outer Clenshaw sweep in x_h . One design lesson from certifying these kernels

deserves record, because it is counterintuitive: for a kernel that feeds a downstream polynomial fit, the smoothness of its error function matters as much as its magnitude. An early candidate for the logarithm kernel, a plain minimax polynomial of the *same* worst-case accuracy as the reduction above, raised the Chebyshev degrees demanded by the table cells by eleven, because its equioscillating error injected a rough component into the quantity being fitted; the smooth-error reduction adds no degree at all. Kernel acceptance was therefore gated on the downstream fitted degrees, not only on the kernel’s own error.

Each kernel has an eight-wide AVX-512 twin and a four-wide AVX2 twin that reproduce the scalar fused-multiply order lane for lane; masked selections use blends whose argument order is mirrored from the scalar ternaries, and per-lane routing decisions are the same integer tests as the scalar router. The only intrinsic without a direct AVX2 analogue is the exponent scaling `scalef` used by the shared exponential; we reproduce it, including its behaviour in the subnormal underflow range, by an exact two-factor power-of-two construction (splitting the integer exponent so that each factor remains a normal number), so the AVX2 path matches both the scalar `ldexp` and the AVX-512 `scalef` across the whole finite range; the equivalence was verified bit-for-bit over seven hundred thousand sample arguments spanning the full reachable range and beyond, down to the underflow threshold.

The batch driver is shaped by what a real workload contains. An end-of-day option book, re-expressed on the out-of-the-money branch (Section 6), is about 90% small-moneyness quotes; a general mixed-maturity batch is therefore dominated by one chart, and the driver bets on that. It streams the (h, c) vector eight (or four) lanes at a time and, for each block, does three things at once: it classifies every lane by evaluating the three seam curves; it evaluates the small-moneyness kernel on *every* lane speculatively; and it stores, with a masked write, only the lanes that classified as small-moneyness. Those are the $\sim 90\%$, written in input order with no sort and no scatter. The minority that classified otherwise is compressed into a small deferred buffer and handed to a conventional counting-sort-by-cell pass over the remaining charts. Because the dominant chart never touches the sort, the mixed-maturity cost per quote falls close to the kernel’s own cost rather than to the cost of routing plus sorting plus gathering every quote individually.

Making the classification itself vectorizable required removing the last library calls from the router. The two seam prices consumed on the hot path, $C(h, 1.70)$ (small-moneyness versus large-volatility) and $\text{expm1}(h)$ (the small-moneyness input), are replaced by frozen low-degree Chebyshev fits, and the wing and central seams are moved onto the shared exp kernel; the scalar entry adopts the identical fits, so the classification is a handful of branchless polynomials that the scalar and SIMD routers evaluate to the same bits, and the route each quote takes is unchanged. Speculation costs the wasted small-moneyness evaluations on the deferred minority, far less than the sorting it removes on the majority; and it is gated: a strided sample of the batch’s moneyness picks the speculative driver only when the small-moneyness share is actually high, and otherwise falls back to the plain counting-sort-by-cell driver, which is faster on the wing- or central-dominated feeds a real book never produces. Both drivers return bit-identical results, so the dispatch changes only speed.

Three further drivers complete the set. A fixed- h surface driver for the volatility-surface case, in which all quotes share one maturity and the h -derived constants are hoisted out of the loop entirely, is the throughput ceiling. A permuted-output variant returns results in bucket order with the permutation, saving the scattered store when the caller can consume sorted output. And a warm-start driver for streaming recalibration takes the previous variance of each (K, T) node as its seed, skips routing entirely, and applies three fixed Householder steps on the exact equation; within its basin (previous $W \leq 40$, volatility moving up to a few percent) it returns the new root to within a few ULP rather than the last bit, because the half-ULP rounding of its working variables is amplified by the deep-wing cancellation ratio, so a caller needing last-bit output re-inverts cold—a documented API contract, not folklore. For the live streaming regime,

where snapshot-to-snapshot moves are sub-percent, a measured two-step variant reaches the same warm floor as three steps (verified for $|\Delta v/v| \leq 1\%$ at every $W \leq 40$; it degrades only from 2% moves at deep W) and is guarded by the size of its own last correction, so a move outside the two-step basin flags itself and falls back to a cold inversion rather than returning a degraded result silently. A thin book-mode wrapper packages the pattern: because exchange strike grids are standardized, the (K, T) node set of a live book is persistent, and the caller carries only the static per-node log-strikes and the previous variances between snapshots—recomputing h under a new per-expiry forward is then one subtraction per node, and the whole book warm-refines with no routing at all. In every driver the deep-wing bucket is split by the wing’s three evaluation regimes so each sub-bucket is lane-uniform, and the wing kernel has both an AVX-512 and an AVX2 twin. Two routes remain scalar by design: the exact at-the-money line $h = 0$ and the rare large- β rescue branch of Section 4.3; both drop to the same non-inlined scalar entry the reference path uses, so the bit-identity statement covers them trivially.

The correctness of the vectorization is checked adversarially rather than assumed. Perturbing a single constant inside the vector kernels alone, by one part in 2^{40} , makes the batch result diverge from the scalar result on precisely the quotes routed through those kernels (more than two thousand of the endpoint quotes in the validation feed) while the unperturbed build shows zero divergences; this confirms that the vector path genuinely executes, rather than silently falling back to the scalar kernels and passing the identity check vacuously. The same check was run independently for the AVX-512 and AVX2 builds.

The bit-identity invariant turns out to constrain the *build*, not only the source, and the constraint is instructive enough to record as a finding. Under GCC’s default C++ contraction mode (`-ffp-contract=fast`) the compiler is free to fuse any multiply–add pair, and it exercises that freedom during middle-end optimization, *after* inlining—so which pairs fuse depends on the inlining context of the surrounding translation unit. We observed the same header, compiled at identical flags into two translation units of one program, produce two floating-point functions that disagree by one to five ULP on about seven percent of deep-wing quotes: a non-inlined entry point compiled to 4,427 instructions in the benchmark unit and 2,932 in a test unit, from identical source. No amount of source-level care defends against this, because the divergence is created by the optimizer’s context-dependent choices; selectively freezing individual products, our first response, is whack-a-mole. The robust discipline is the opposite default: compile with contraction *off*, and write every fusion one wants as an explicit `fma`. Since the accuracy-critical chains here were written that way from the start, the flag costs nothing measurable, and it upgrades determinism from a per-build property to a portable one. On every validation feed of Section 6, the AVX-512, AVX2 and non-SIMD builds, compiled with either GCC 11 or Clang 14, return bitwise-identical doubles for every quote, scalar and batch paths alike—verified by byte comparison of the full output vectors, not by summary statistics. In production terms the invariant buys more than testability: a risk system replaying a batch, re-pricing a single quote from a batch, or reconciling results across heterogeneous nodes and compilers obtains identical doubles in every case, which removes a whole class of reconciliation noise that iterative solvers with data-dependent iteration counts can exhibit.

The input contract. A reference implementation is defined as much by what it does with bad inputs as by its accuracy on good ones, because a live feed delivers crossed quotes, stale prices and convention violations as a matter of course, and the worst possible response is a plausible-looking number. The entry points therefore implement a *total* contract: every representable pair (h, c) returns either a machine-precision result or a quiet NaN, and a checked variant reports a machine-readable status alongside—at or below intrinsic ($c \leq 0$), at or above the forward bound ($c \geq 1$), convention violation ($h < 0$ or NaN input), the out-of-domain deep corner of Section 4.3, and a near-saturation advisory for $v > 8$, where the $1 - c$ representability bound of Section 2 limits what any double-precision inverter can resolve; in that last case the best-effort result is

still returned. A convenience wrapper accepts the industry-convention inputs (forward, strike, undiscounted price, expiry, call/put flag), performs the in-the-money to out-of-the-money parity flip internally, and returns the annualized volatility with the same status, which removes the most common integration error, a wrong normalization, from the caller’s responsibility.

6 Numerical results

Oracle and test sets. The reference is an independent `mpmath` inverter at 40 significant digits that inverts the given double-precision price c by a bounded log-space Newton iteration (not `findroot`, not a round trip). Accuracy is the relative error $|\hat{\sigma} - \sigma_{\text{oracle}}|/\sigma_{\text{oracle}}$, and we also report the worst-case error in units in the last place (ULP) of double precision. We score on a 150×110 grid over $h \in [0.02, 16]$, $\sigma\sqrt{T} \in [0.02, 8]$ (16,039 feasible points, the “regular” set), on a hand-built “stressed” set of deep-wing, near-intrinsic and small- h deep-OTM points, and on a 3,511-point chart-stress feed constructed to exercise every routing region and seam. The competitor is the reference C++ implementation of Jäckel (2015), called through its normalized entry point with $\beta = ce^{-h/2}$; we verified on the extreme cases that this normalized call agrees with the library’s direct price interface, so no artefact of the calling convention enters the comparison. One methodological trap deserves emphasis because it silently corrupts comparisons of this kind: scoring an inverter against the volatility that *generated* a test price, rather than against an independent inversion of the price actually given, adds the forward pricer’s own cancellation error to every method’s account. On our feeds that construction error reaches tens to hundreds of ULP in the wings and is statistically identical across all methods tested, including the reference; all figures below are free of it.

Hardware, compilers and timing protocol. All measurements were taken on an Intel Core i5-1145G7 (Tiger Lake, AVX-512-capable, four cores) under Linux (Ubuntu 22.04, WSL2), with GCC 11.4 at `-O3 -ffp-contract=off -fno-fast-math` and the instruction set selected per build (`-march=native` for AVX-512, `-mavx2 -mfma -mno-avx512f` for AVX2, both SIMD sets disabled for the scalar build); the Clang 14 cross-check uses the same flags. The timing protocol pins the process to a fixed core, takes medians across no fewer than twenty runs of ten repeats each, and guards against dead-code elimination with a volatile accumulator. Contraction is disabled for the reproducibility reasons of Section 5—every intended fusion is an explicit `fma`—and `-ffast-math` is deliberately avoided because it would break the scalar/SIMD bit-identity. All accuracy and bit-identity results are host-independent and final. The development host is a virtualized laptop rather than a quiet benchmarking machine, and its absolute nanosecond figures fluctuate across sessions by factors that dwarf the effects being measured; the definitive timing table is therefore taken on a dedicated, frequency-pinned host under the same protocol, and the development host is used only to establish orderings.

Accuracy. Table 2 and Figure 2 give the result. The routed inverter holds below 10^{-15} at *every* point of the feasible domain, with a worst relative error of 7.9×10^{-16} (5 ULP) and a median at the level of a single rounding. On the interior and the small- h region it is indistinguishable from Let’s Be Rational, confirming that the table construction reaches the same last-bit accuracy as a state-of-the-art iterative solver. The two methods separate only at the extremes (Figure 3). In the near-intrinsic right endpoint ($c \rightarrow 1$, $\sigma\sqrt{T} \rightarrow 8$) the reference solver loses up to 867 ULP where the routed inverter stays at 3 ULP, because the cancellation-free transformed price $\tilde{c}$ of Section 4 avoids the loss that $1 - c$ incurs. In the deep wing the two methods receive inputs of different quality: the reference’s $\sqrt{FK}$ -normalized price $\beta = ce^{-h/2}$ is smaller than the forward-normalized c by the factor $e^{-h/2}$, so at large h it reaches the subnormal range, where doubles carry fewer significant bits, several hundred orders of magnitude before c does; its accuracy there reflects that input conditioning rather than its iteration. The routed inverter,

Region	Routed table inverter			Let’s Be Rational		
	max rel	RMS rel	max ULP	max rel	RMS rel	max ULP
Central	4.3×10^{-16}	1.2×10^{-16}	3	5.0×10^{-16}	1.2×10^{-16}	3
Left (small h)	6.7×10^{-16}	2.3×10^{-16}	4	8.3×10^{-16}	2.4×10^{-16}	5
Right (large v)	4.3×10^{-16}	1.0×10^{-16}	3	1.1×10^{-13}	4.8×10^{-15}	867
Wing (deep OTM)	7.9×10^{-16}	9.6×10^{-17}	5	degrades; see text		
All	7.9×10^{-16}	1.0×10^{-16}	5	—	—	—

Table 2: Relative error in σ against a 40-digit oracle on the regular set (16,039 points). The routed inverter has *no* point above 10^{-15} ; its worst error is 7.9×10^{-16} (5 ULP). Let’s Be Rational matches it on the central and small- h regions but degrades in the near-intrinsic right endpoint and the deep wing (text).

whose wing chart works on the logarithm of the intact price, inverts prices down to 10^{-320} at machine precision. The stressed set tells the same story: 0 routed points above 10^{-15} (worst 6.9×10^{-16}) against a 3.4×10^{-14} right-endpoint tail for the reference. Across all validation feeds combined, including the chart-stress feed, the worst error the routed inverter has exhibited anywhere is 8.3×10^{-16} ; no feed has produced a point above 10^{-15} . The seam crossings between charts were additionally sampled densely on both sides; no error excursion occurs at any of them. We note for honesty that this was not automatic: dense seam verification during development exposed two seam placements whose corners violated the gate, and both were corrected (one by moving the small- h left-right seam to $v = 1.70$, one by extending the wing crossover), which is the intended operation of an oracle-gated design rather than a caveat about the final result.

Honesty about where the accuracy advantage does and does not matter is essential here. The two regimes in which the reference degrades—the near-intrinsic upper endpoint and the subnormal-price wing—are extreme: they require normalized prices within a rounding of one, or below 10^{-40} . A tradeable option book reaches neither. Inverting an independent 30-digit oracle on 1,150 *actual* projected prices sampled from the 2024 S&P 500 book (Section 6), stratified to over-weight the deep-cheap and seam-band quotes, the routed inverter has no point above 10^{-15} (worst 8.5×10^{-16})—and so does the reference. On real market data, in other words, both solvers are at machine precision, and the case for the present method there rests on throughput and determinism, not on accuracy. The accuracy advantage is real but it is insurance: it guarantees the same last-bit behaviour on the theoretical, stressed, and finer-tick inputs that a robust production system must also survive, without the reference’s two localized failure modes. We state this rather than lean on a heatmap whose hot corners a live feed never visits.

Edge cases. Table 3 isolates individual extreme inputs, with the exact double-precision price each method was given and the relative σ error each returned. The routed inverter is exact or within four ULP on every row, including prices in the subnormal range near 10^{-321} , the formerly excluded small- h deep-OTM corner, the near-intrinsic upper endpoint, and the feasibility pinch at $h = 16$. The reference solver’s two visible losses have different characters and should be read differently. Near the intrinsic value its error (10^{-14} scale) is algorithmic, the cancellation discussed in Section 2. In the subnormal wing its input $\beta = ce^{-h/2}$ has already lost significance before the algorithm starts, so the error there (10^{-6} to 10^{-4}) reflects the conditioning of its chosen normalization rather than its iteration; the routed inverter is immune because its normalization keeps the small price intact and its wing chart works on the logarithm of the price. At the two most extreme prices of the regular grid, below 10^{-320} , the β input underflows to exactly zero and the reference necessarily returns zero volatility, while the routed inverter still recovers σ to the correctly rounded last bit; prices this small are admittedly of more diagnostic than trading interest, but they mark where each formulation’s validity actually ends.

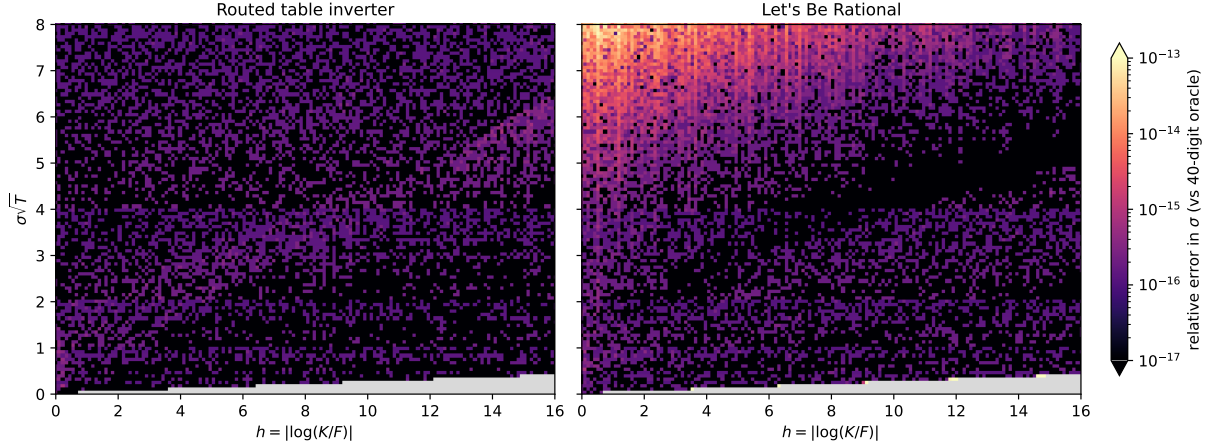


Figure 2: Relative error in σ against the 40-digit oracle over the $(h, \sigma\sqrt{T})$ domain. The routed table inverter (left) is uniformly at machine precision; Let’s Be Rational (right) degrades in the near-intrinsic upper-left corner and, more mildly, in the deep wing. Grey marks prices that round to 0 or 1 in double precision.

h	$\sigma\sqrt{T}$	price	regime	this method	Let’s Be Rational
14.8	0.386	$c = 9.9 \times 10^{-321}$	wing	0	1.4×10^{-4}
3.56	0.0932	$c = 2.2 \times 10^{-321}$	wing	0	3.9×10^{-6}
8	0.90	$c = 1.5 \times 10^{-18}$	wing	1.2×10^{-16}	0
0.29	0.02	$c = 9.6 \times 10^{-51}$	corner	0	0
10^{-4}	1.0	$c = 0.38$	left	0	0
0	6.4	$c = 0.9986$	ATM	5.6×10^{-16}	1.4×10^{-16}
0.5	7.9	$1 - c = 1.0 \times 10^{-4}$	right	1.1×10^{-16}	5.6×10^{-16}
1.2	7.93	$1 - c = 1.3 \times 10^{-4}$	right	1.1×10^{-16}	1.3×10^{-14}
16	7.5	$c = 0.93$	right	1.2×10^{-16}	1.2×10^{-16}

Table 3: Relative error in σ against the 40-digit oracle on hand-picked extreme inputs; an entry of 0 means the returned double equals the correctly rounded oracle value. The reference solver’s subnormal-wing losses reflect the conditioning of its $\sqrt{FK}$ normalization (text); its near-intrinsic losses are the $1 - c$ cancellation.

Coverage and bit-identity. On a broad random feed of 120,000 quotes ($\sigma\sqrt{T}$ up to 8, h up to 16.5) the four charts cover 100% of the feasible domain. Across the scalar, AVX2 and AVX-512 builds and all batch drivers (fixed- h surface, mixed- h grid, permuted-output, warm-start), the batch result equals the scalar result on every one of the 120,000 quotes, with zero mismatches; the same holds on the 16,039-point accuracy grid. Beyond within-build identity, the full output vectors of the three instruction-set builds under GCC and of the AVX-512 and AVX2 builds under Clang were compared byte for byte on the chart-stress feed: all five binaries return identical doubles for every quote, in both the scalar and the batched path. The accuracy of Table 2 is therefore simultaneously the accuracy of every batch path, every instruction set, and both compilers.

What a real book exercises. Before reporting throughput it is worth establishing, from data rather than intuition, which charts a realistic workload actually hits. We took one full year (2024) of end-of-day S&P 500 index option quotes from a standard vendor file—1.85 million with positive volume and open interest—and applied a tradeability screen: a live, uncrossed two-sided market, mid price at least five cents, relative bid–ask spread at most 25%, open interest at least 100 contracts and volume at least 10, and at least a week to expiry. This leaves

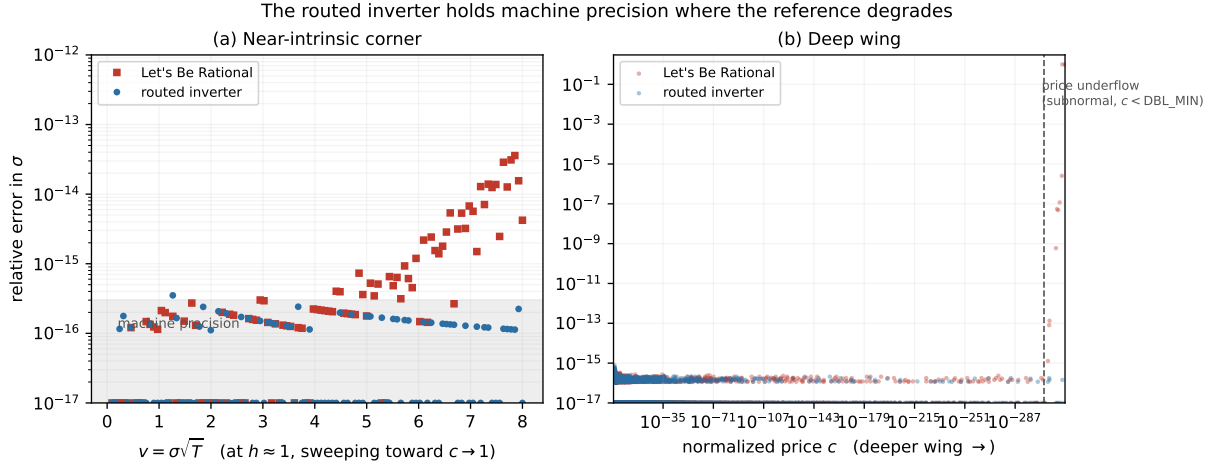


Figure 3: Relative error in σ against the 40-digit oracle in the two regimes where the reference degrades. **(a)** Near the intrinsic value: at fixed $h \approx 1$, as v grows and the normalized price $c \rightarrow 1$, Let’s Be Rational climbs toward 10^{-13} from the $1 - c$ cancellation while the routed inverter stays at machine precision. **(b)** In the deep wing, error against the price magnitude c over the whole wing chart: both solvers hold machine precision across three hundred orders of magnitude, but once the price is subnormal ($c < \text{DBL_MIN}$) the reference’s reduced variable $\beta = ce^{-h/2}$ underflows and it loses precision, whereas the routed inverter, working in $c = \text{price}/F$, does not.

641,072 tradeable quotes. Forwards are the vendor’s own, matched to each quote by expiry *and* settlement convention (the file carries distinct AM- and PM-settlement forwards, differing by about half an index point), and discount factors come from the vendor zero curve; we do not rely on a parity regression, which we found can bias the implied forward by up to several tenths of a percent at individual expiries where deep in-the-money premiums dominate the fit. Every surviving quote—call or put, in- or out-of-the-money—is then folded onto the single out-of-the-money-call branch by put–call parity, exactly the reduction the inverter itself uses; 15.8% of quotes are in-the-money and reach the branch through the parity projection. Routing the resulting (h, c) pairs gives: small-moneyness chart 90.0%, deep wing 5.1%, central table 4.9%, large-volatility chart 0.0% (Figure 4). Three facts deserve emphasis. The wing, though a single-digit fraction, is a real and permanent feature of a live book—about one tradeable quote in twenty, and 98% of those downside puts, the crash-skew tail firing the wing route on absolute cheapness at small moneyness—which is what motivated vectorizing it rather than leaving it scalar. The large-volatility chart never fires on end-of-day equity-index data (no traded quote is that expensive in normalized terms); it earns its place as coverage, not throughput. And the maximum log-moneyness observed all year is $h = 3.4$, far inside the design domain: the deep tail the wing chart handles down to 10^{-320} is exercised by theoretical and finer-grained prices, not by vendor end-of-day files. The uniform broad box used for coverage stress, by contrast, is 78% wing—a distribution no traded book resembles—so throughput is reported on a market-realistic feed, built by resampling 30,000 quotes from this tradeable population, which reproduces the live route mix.

Throughput. Table 4 and Figure 5 report median nanoseconds per quote for the reference solver, the routed inverter’s scalar entry, and its batch drivers on both instruction sets. So that no method is flattered by the harness, the market-feed row is timed *all-inside*: every method starts from the raw (h, c) pair within the timed loop, so the reference pays its $\beta = ce^{-h/2}$ input transform and our scalar entry builds its per-maturity context, and nothing is precomputed for anyone. The surface rows, by contrast, isolate the kernels by hoisting the shared per- h constants, as any surface caller would. Three patterns stand out. On chart-pure fixed- h surfaces the batch

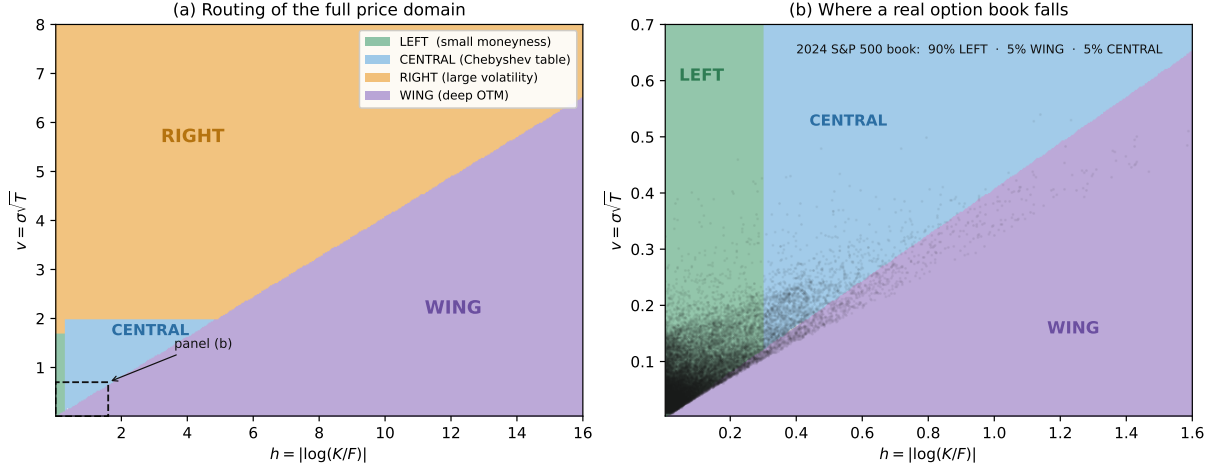


Figure 4: Routing of the normalized price domain. **(a)** The feasible (h, v) plane, $h = |\log(K/F)|$ and $v = \sigma\sqrt{T}$, partitioned into the four charts of Sections 3–4; the boundaries are fixed by the maturity-independent branch-point geometry of the inverse map, not tuned. **(b)** A zoom into the near-the-money corner (boxed in (a)) with an entire year of tradeable 2024 S&P 500 end-of-day quotes overlaid: about 90% of a real book falls in the small-moneyness (LEFT) chart and a further 5% in the deep wing, while the large-volatility (RIGHT) region that dominates the plane by area carries essentially none.

is several times faster than the reference: $9.5\times$ on the central table, $5.1\times$ on the small-moneyness chart, and $2.6\times$ on the compute-heavy large-volatility endpoint (AVX-512; AVX2 recovers 7.7, 3.3 and $1.4\times$). This is the direct consequence of a fixed, branch-light operation count—eight or four quotes advancing through the identical polynomial sweep in lockstep, where a data-dependent iteration cannot. On the *market-realistic feed*, the workload that matters, the speculative driver of Section 5 inverts a mixed S&P 500 book at 77 ns per quote against the reference’s 226, a factor of 2.9; on AVX2 it is 108 against 225, a factor of 2.1. That the reference here is timed with its own per-quote transform inside the clock is what makes this an honest head-to-head, and the small-moneyness chart carrying 90% of the book at close to its kernel cost is what carries the result. The one regime in which the reference is faster per quote is the deep wing, whose resurgent evaluator carries more arithmetic than a Householder iteration. On a real book only about one quote in twenty is a wing quote, so this never dominates the aggregate; a synthetic feed deliberately weighted toward the deep wing—which no traded book resembles—is the sole distribution on which the reference leads, and the adaptive dispatcher routes such a feed to the plain sort-then-batch driver rather than the speculative one.

One throughput property is invisible in medians but operationally significant: the cost per quote is *deterministic*. Every chart executes a fixed operation count (a fixed-degree polynomial sweep, or a fixed number of refinement steps—six Newton in the wing, three Householder at the large-volatility endpoint), so the routed inverter has no iteration-count distribution and no data-dependent tail latency; the slowest quote of a batch costs the same as the median quote of its chart. Solvers that iterate to a tolerance necessarily carry a tail, however short, and hedging pipelines that wait on the last quote of a batch inherit it.

Streaming re-inversion. A live system rarely inverts cold twice: the persistent node set of Section 5 means every snapshot after the first starts from the previous one. On the market feed, seeding each node with its previous variance after a half-percent move, the guarded two-step warm batch completes in 0.52 times the cold speculative driver’s time on the same feed and host—40 ns per quote against the cold driver’s 77 and the reference’s 226, a $5.7\times$ margin over the reference on the workload live systems actually run—and, having no routing and no deferred

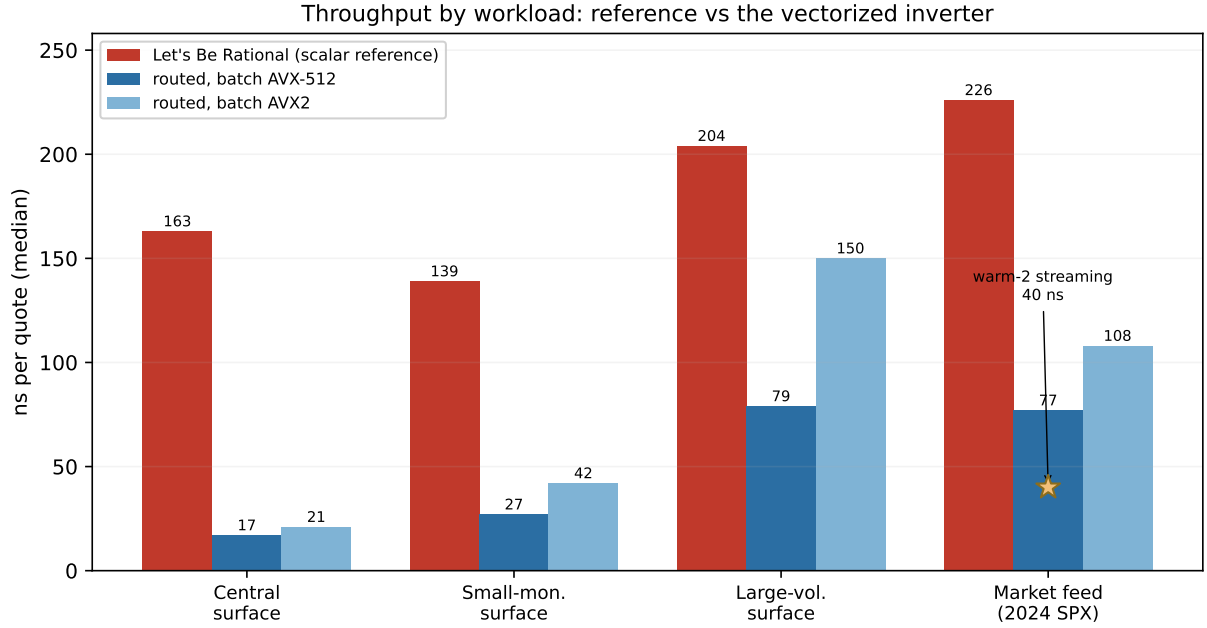


Figure 5: Median nanoseconds per quote by workload (the data of Table 4), for the scalar reference solver and the routed inverter’s two vectorized batch paths (AVX-512 and AVX2). The batch paths are several times faster than the reference on the chart-pure surfaces and on the market-realistic feed. The star marks the warm streaming steady state on the market feed (40 ns, AVX-512).

drain, its cost is flat in batch size (Figure 1), whereas the cold driver’s sort-and-drain overhead only amortizes over batches of a few thousand. The honest counterpart concerns the single isolated quote: inverted scalar and cold, one quote at a time, the reference solver remains faster than our scalar entry, whose per-quote context and routing were designed to amortize across a batch. The streaming answer to a single ticked node is therefore the guarded warm refinement of its previous value, not a cold one-off; a caller who genuinely needs isolated cold scalar inversions at minimum latency is the one workload for which the reference remains the better tool.

Relation to other methods. The natural axes of comparison are accuracy class, domain of validity, and unit of work. The explicit formulas of Li (2008), Stefanica and Radoičić (2017) and Xia and Cui (2018) are the fastest possible single evaluations and vectorize trivially, but they are approximations valid on bounded regions with accuracy far from the last bit; in the present architecture their natural role is as seeds, and the rational quantile seed of Section 4.2 plays exactly that part. Matić et al. (2018) precompute a polynomial table over moneyness and price, as we do, but target an accuracy of 10^{-7} ; the present table reaches 10^{-15} from a comparable byte budget because its coordinates and degrees are fixed by the branch-point analysis of Section 3 rather than fitted empirically, and no head-to-head timing is offered here since no reference implementation of that method was available. Jäckel (2015) remains the accuracy standard, and Table 2 shows the present method matches it everywhere both are at machine precision, while extending that precision into the two extreme regimes and changing the unit of work from a branchy scalar iteration to a vectorizable fixed-cost evaluation. The concurrent work of Le Floc’h and Healy (2026) optimizes the complementary objective: single-quote latency of a seed-plus-refinement solver, with a controlled accuracy relaxation. The two designs meet in their shared insistence on tail-stable formulations, and differ in what they spend the accuracy budget on; a direct benchmark must await a common harness and is not attempted here.

Workload	Let’s Be Rational	Routed inverter, ns per quote		
		scalar	batch AVX-512	batch AVX2
Central surface, $h=1$	163	70	17	21
Small-moneyness surface, $h=0.2$	139	182	27	42
Large-volatility surface, $h=1$	204	666	79	150
Market feed (2024 SPX, actual prices)	226	566	77	108

Table 4: Median nanoseconds per quote (five repetitions, quiet Intel Core i5-1145G7, Linux, GCC 11.4, `-O3 -ffp-contract=off -fno-fast-math`), on chart-pure fixed- h surfaces of 4096 quotes, on the market-realistic feed of 30,000 actual projected prices resampled from the tradeable 2024 S&P 500 population (route mix 90/5/5/0 percent across left/wing/central/right). The market row is timed all-inside (each method pays its own per-quote input transform inside the loop); the surface rows hoist the shared per- h constants. The batch output is bit-identical to the scalar entry on every driver and instruction set (zero mismatches, verified on both the 120,000-point coverage grid and the market feed). Bold marks where the batch beats the reference. Inter-repetition spread was under one percent on every cell.

Limitations. Four boundaries of the claims deserve collection in one place. First, the batch speedups of Table 4 are largest on AVX-512; the AVX2 four-wide twins recover roughly half on the fixed-maturity surfaces, though the market-feed advantage ($2.1\times$) survives because every chart including the wing has an AVX2 twin, and hardware without SIMD falls back to the scalar path (still correct, still gated, still bit-identical, not accelerated). Second, the machine-precision domain has an explicit edge: the wing tables cover $\beta = h^2/16 \leq 16.4$, i.e. $h \leq 16.2$ (strikes beyond 10^7 times the forward), the rescue branch of Section 4.3 certifies the band beyond it for $W \lesssim 40$, and the still-deeper corner returns a flagged non-result; for $v > 8$ the returned volatility carries the input-representability floor of Section 2, which no double-precision interface can circumvent. Third, absolute timings are hardware-specific and a deployment decision should rest on a re-measurement on the target host; the reported figures are medians of five repetitions on a quiet machine with under one percent inter-repetition spread, but the accuracy and bit-identity results, being host-independent, are the load-bearing claims. Fourth, the exhaustive grid validation domain is $\sigma\sqrt{T} \leq 8$, $h \leq 16.5$; the wing evaluator is itself validated on prices to 10^{-320} , but the grid evidence of Figure 2 stops at the stated box.

Reproducibility. The oracle scripts, the accuracy and bit-identity harnesses, the benchmark driver, the market-feed sample and the frozen table are provided as supplementary material, with build lines for all three instruction-set targets; Appendix C lists the artifacts. A golden-vector verifier (oracle-checked $(h, c) \rightarrow w$ triples spanning every chart and seam) lets an adopter confirm, in one command after building, that their compiler and hardware reproduce both the accuracy gate and the bit-identity invariant. Every number in this section is regenerated by the harnesses from the shipped sources.

7 Conclusion

Recovering implied volatility to machine precision is a solved problem for a single scalar quote; what has been missing is a method that reaches that precision through an evaluation that also vectorizes over the whole feasible domain, so that the batched surface and grid workloads which dominate calibration and pricing run at hardware throughput. The routed table inverter presented here supplies one. By representing the interior in the transformed variable $W = h^2/2w$ on a bivariate Chebyshev table whose size is fixed by the maturity-independent location of the inverse map’s branch points, and the endpoints by three analytically matched charts whose

special functions reduce to a single two-parameter family, it resolves every feasible quote by direct evaluation. Routing by IEEE-754 bit fields into branchless kernels shared between the scalar and SIMD paths makes the batched result bit-for-bit identical to the scalar result, a property we verify exhaustively and that extends, under the contraction discipline of Section 5, across instruction sets and compilers: five distinct builds return identical doubles on every validation quote. Against a 40-digit oracle no point of any validation feed exceeds 10^{-15} relative error in volatility, with a worst observed error of 8.3×10^{-16} : the method matches Let's Be Rational on the interior and holds machine precision in the near-intrinsic and deep-wing regimes where the reference degrades. All four charts vectorize, every representable input receives a machine-precision result or an explicitly flagged non-result, and a speculative streaming driver built on the empirical dominance of the small-moneyness chart inverts a market-realistic S&P 500 book at 2.1–2.9 times the reference's throughput and fixed-maturity surfaces at 3–9 times; in the streaming regime that live systems actually run, the guarded warm refinement of a persistent book roughly doubles the market-feed rate again. On that tradeable book both solvers are at machine precision, so on real data the method's case is throughput and determinism; its accuracy advantage is insurance for the extreme inputs—near-intrinsic prices and the subnormal wing—that a robust deployment must survive but a live feed does not visit.

Two boundaries bound the claims. The peak batch speedup is on AVX-512; the AVX2 four-wide twins recover roughly half on the surface workloads, though the market-feed advantage ($2.1\times$) survives because the wing now has an AVX2 twin. And the machine-precision domain, while covering strikes beyond 10^7 times the forward, is finite and explicitly flagged at its edge rather than extrapolated. The natural extensions are a Bachelier (normal-volatility) analogue, for which the same routed-chart architecture applies with the normal density in place of the log-normal, and a data-parallel GPU port, for which the fixed-cost, branch-light kernel structure is well suited.

A The hot path

The complete scalar evaluation, which the batch drivers reproduce bucket-wise, is the following. All constants are frozen doubles; $\text{bits}(\cdot)$ denotes IEEE-754 bit extraction; $\log_2^*$ is the shared logarithm kernel of Section 5.

Routing (given $h \geq 0$ and $c \in (0, 1)$):

1. if $h = 0$: solve the exact ATM relation $c = \text{erf}(\sigma/2\sqrt{2})$ (quantile seed, two guarded corrections); return.
2. if $c < c_{\text{wing}}(h)$ (one Chebyshev in h): WING.
3. if $h < 0.3$: LEFT if $c \leq c_{\text{tl}}(h)$ (a frozen degree-8 Chebyshev fit of $C(h, 1.70)$, so the test is a polynomial, not a price evaluation), else RIGHT.
4. if $h \leq H_{\text{box}}$ and $c \leq c_2(h)$: CENTRAL; else RIGHT.

Central (table): band from $\text{bits}(h)$; octave and sub-cell from $\text{bits}(c)$; $x_\lambda = \alpha_{\text{cell}} \log_2^*(m) + \beta_{\text{cell}}$ with m the mantissa of c ; W by the bivariate Clenshaw sweep (4); return $w = h^2/(2W)$.

Left: $\rho = c/\text{expm1}(h)$; $A = B^{-1}(\rho)$ (three-regime Chebyshev); $s = h/A$; $\sigma = \sigma_0(\kappa s) + h^2 V_2(s) + h^4 V_4(s) + h^6 R(s, h)$; return $w = \sigma^2$.

Right: $\bar{g} = \frac{1}{2}(1-c)e^{-h/2}$; $x_0 = -\Phi_{\text{seed}}^{-1}(\bar{g})$ (rational seed); Mills ratio $m_0 = \sqrt{\pi/2} \text{erfcx}(x_0/\sqrt{2})$; ladder corrections $x = x_0 + h^2 x_2 + h^4 x_4$ with x_2, x_4 polynomial in $1/x_0, m_0$; then exactly three Householder-3 steps on (9), with f/g' from (11) and the closed-form ratios $g''/g' = -x + h^2/(4x^3)$, $g'''/g' = (g''/g')^2 - 1 - 3h^2/(4x^4)$, prices evaluated in whichever of $c, 1-c$ is small; return $w = (2x)^2$.

Wing: if $h^2/16 > 16.4$: rescue branch (Φ -inversion seed, three fixed-point corrections, three Householder steps; quiet NaN beyond its certified band). Otherwise $\tilde{L}$ from $\log c$ (subnormal-safe $\log_2^*$) and h ; regime (piece A / piece B / series) once from $\tilde{L}$; exactly six Newton steps on $W = \tilde{L} - \frac{3}{2} \log W + \log H(W)$ with $\log H$ and its analytic derivative from the fitted $S(u, \beta)$ (or the series); return $w = h^2/(2W)$.

Batch drivers: a strided moneyiness sample selects one of two paths. *Speculative* (small-moneyiness-heavy feeds, the live case): stream eight/four-wide; classify each lane by the frozen seam polynomials; evaluate the small-moneyiness kernel on every lane; masked-store the small-moneyiness lanes in input order; compress the rest into a deferred buffer. *Sort-then-batch* (the deferred buffer, and minority-dominated feeds): classify, counting-sort by cell, chart and wing regime, and evaluate each bucket eight/four-wide. Both are bit-identical to the scalar entry, so the dispatch changes only speed; remainders, the rescue branch and the ATM line go through the scalar entry unchanged.

B Table generation and certification

The offline pipeline that produces the frozen constants is as important to the accuracy claim as the runtime, so we record it.

1. *Truth generation.* For each cell, Chebyshev nodes in (x_h, x_λ) are mapped to exact double-precision prices, and each price is inverted by an arbitrary-precision solver (no fewer than 25 significant digits, price residual driven below 10^{-18}); the transformed truth $W = h^2/(2w)$ is recorded.
2. *Fitting.* Tensor Chebyshev coefficients are computed per cell; degrees are the smallest passing the gate below, capped at (13, 16).
3. *Gating.* An independent 40-digit oracle, sharing no code with the generator, re-inverts a dense sample of each cell including its boundaries and the sub-cell seams; the cell is accepted only if the relative σ error is below 10^{-15} at every sample. Failing cells are refit at higher degree or split. The same gate certifies every auxiliary kernel ($\log_2^*$, σ_0 , B^{-1} , V_2 , V_4 , erfcx , the exponential, the residual R), with the additional downstream-degree criterion of Section 5.
4. *Freezing.* Accepted coefficients are emitted as exact hexadecimal doubles into a header; the runtime contains no fitted constant that did not pass the gate.

Certified enclosures for the endpoint functions are available independently of the gate through the Stieltjes representation of Section 4.4, whose continued-fraction convergents bound B , B_n and B^{-1} from alternating sides.

For the benefit of implementers we also record the design alternatives that the gate *rejected*, since each looks attractive on paper. A transcendental-free table in the raw mantissa coordinate (routing and abscissa both from bit fields, no logarithm at all) fails on degree: the h -independent branch-point distances of Section 3 pin the Bernstein parameter and roughly double the degree per cell, blowing the byte budget. A conformal coordinate adapted to the annulus fails the same way, more severely. Evaluating the cells in the monomial basis with Estrin’s scheme, instead of Chebyshev with Clenshaw’s, fails the precision gate outright at the required degrees. An equal-accuracy minimax replacement for the logarithm kernel fails through the downstream-degree effect described in Section 5. Fitting the wing’s $\log H$ directly, or its unweighted peeled residual, demands degrees past thirty-six where the σ -weighted form needs twenty-six; and a $(1/W, \beta/W)$ chart for the same fit, natural-looking because $\beta/W = v^2/8$ is bounded, brings no degree reduction because the hard structure at the small- W edge is resurgent rather than an artifact of coordinates. Finally, one accepted choice had to be revised by a denser gate: five

wing Newton steps passed a sparse step-count de-risk yet left six points of the dense regular grid above 10^{-15} (worst 1.2×10^{-14}), all at the high- β , low- W corner where convergence is slowest; the fixed count is six, and the incident is a reminder that step-count de-risking must sample the corner of slowest convergence, not the typical point. Each rejection was measured, not argued.

C Artifacts

The supplementary material contains: the header-only C++17 implementation (runtime, frozen tables, wing evaluator, checked API); the oracle generators (`mpmath`, 40 digits) for the regular, stressed and edge sets; the golden-vector verifier and its oracle file (the one-command build check of Section 6); the accuracy harness producing Table 2, Table 3 and Figure 2; the bit-identity and poison-test harnesses; the market-feed sample (30,000 resampled (h, v) pairs from the cleaned 2024 population); and the throughput benchmark, including the build lines for the AVX-512, AVX2 and scalar targets used in Section 6. The reference implementation of Jäckel (2015) is publicly available from the author’s site and is linked unmodified. One portability note: a code-generation defect in one Windows GCC toolchain (misaligned vector spills in a heavily spilled function, affecting AVX-512 and AVX2 builds alike and unrelated to this code’s logic) can crash the mixed-grid driver on that platform; the artifacts document the affected configuration, and all results here are from Linux, which is unaffected.

References

- Cody, W. J. (1969). Rational Chebyshev approximations for the error function. *Mathematics of Computation*, 23(107):631–637.
- Jäckel, P. (2006). By implication. *Wilmott*, pages 60–66. November issue.
- Jäckel, P. (2015). Let’s be rational. *Wilmott*, 2015(75):40–53.
- Le Floc’h, F. and Healy, J. (2026). Implying volatility: How fast can we go? [arXiv:2605.29102](https://arxiv.org/abs/2605.29102).
- Li, M. (2008). Approximate inversion of the Black–Scholes formula using rational functions. *European Journal of Operational Research*, 185(2):743–759.
- Matić, I., Radoičić, R., and Stefanica, D. (2018). A PDE method for estimation of implied volatility. Technical Report 3264356, SSRN Electronic Journal.
- Stefanica, D. and Radoičić, R. (2017). An explicit implied volatility formula. *International Journal of Theoretical and Applied Finance*, 20(7):1750048.
- Xia, Y. and Cui, Z. (2018). An exact and explicit implied volatility inversion formula. *International Journal of Financial Engineering*, 5(3):1850032.